

02 矩阵

2.4 矩阵乘法

形状匹配、逐元素点积与NumPy的@运算符



鸢尾花书·《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1

理解矩阵乘法：两个矩阵相乘生成一个新矩阵，记作 $C = AB = A @ B$

2

掌握形状匹配规则：左矩阵列数须等于右矩阵行数，乘积形状由左行、右列决定

3

掌握用NumPy执行矩阵乘法：@运算符、np.matmul()、np.dot() 三种写法

4

理解乘积元素的计算方法：由对应行向量与列向量的点积得到

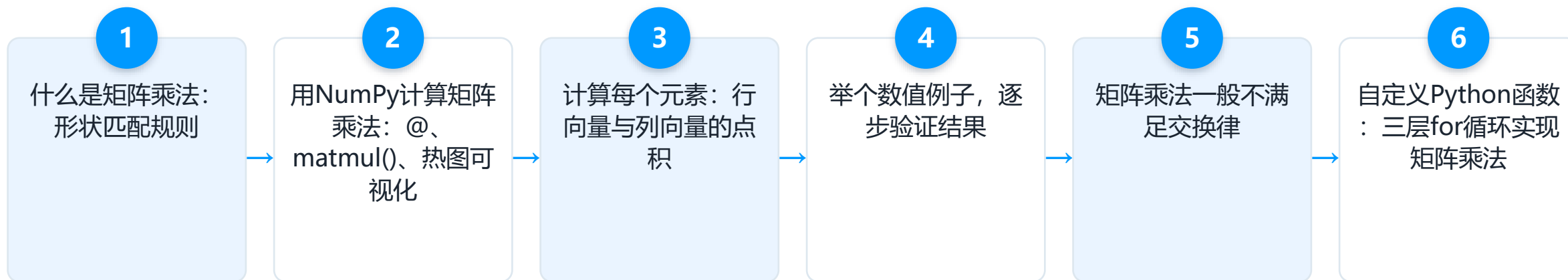
5

认识矩阵乘法可能退化为标量： 1×1 矩阵的乘积结果

6

理解矩阵乘法一般不满足交换律，即 $AB \neq BA$

本节知识脉络



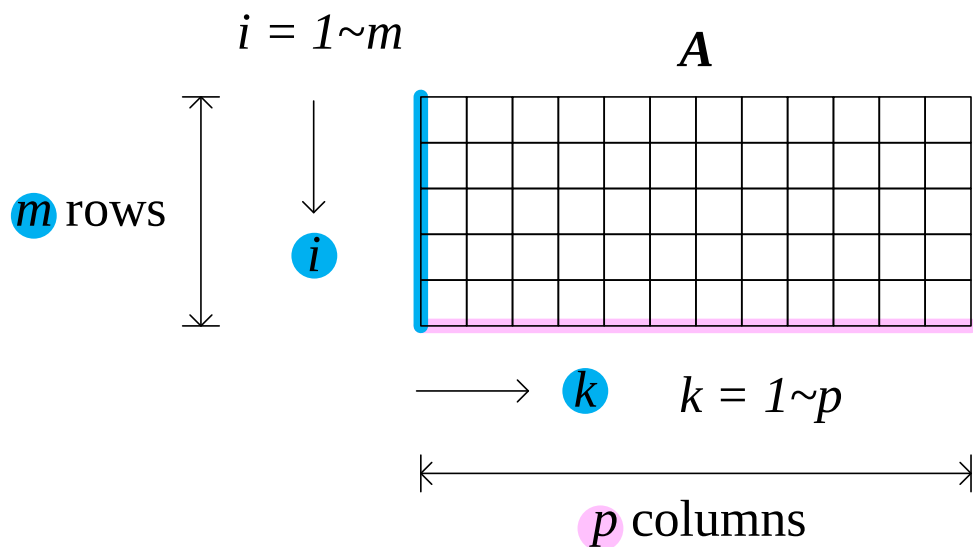
什么是矩阵乘法？

矩阵乘法 (matrix multiplication) 将两个矩阵相乘，生成一个新的矩阵

- 矩阵乘法可能是线性代数中最重要的运算，没有之一；请大家特别注意形状匹配
- 有些时候矩阵乘法的结果也可以是 1×1 矩阵，我们将其视作标量

图1 • 矩阵A和B的形状

- 矩阵**A**的形状为 $m \times p$ ，即有 m 行、 p 列；矩阵**B**形状为 $p \times n$ ，即有 p 行、 n 列
- **A**的列数与**B**的行数必须相等，都是 p ，这是矩阵乘法成立的前提



$$\begin{array}{ccccc}
 \mathbf{A} & @ & \mathbf{B} & = & \mathbf{C} \\
 m \times p & & p \times n & & m \times n
 \end{array}$$

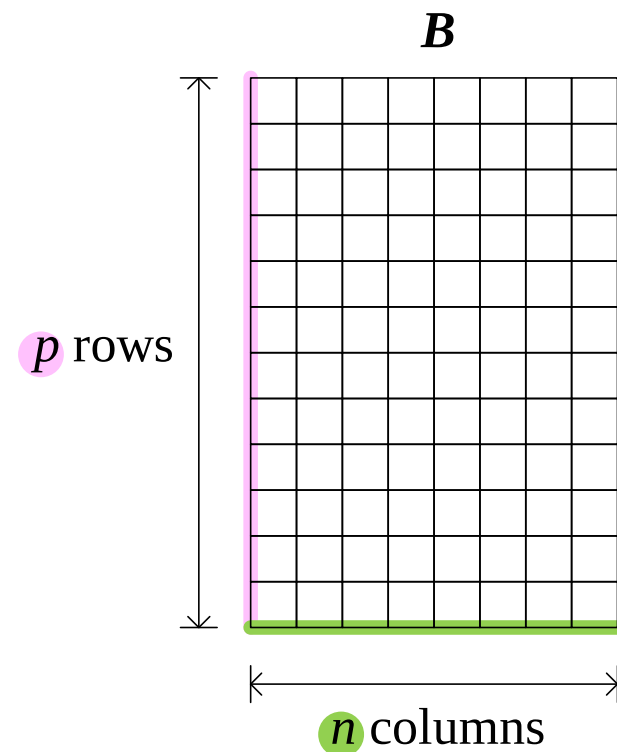
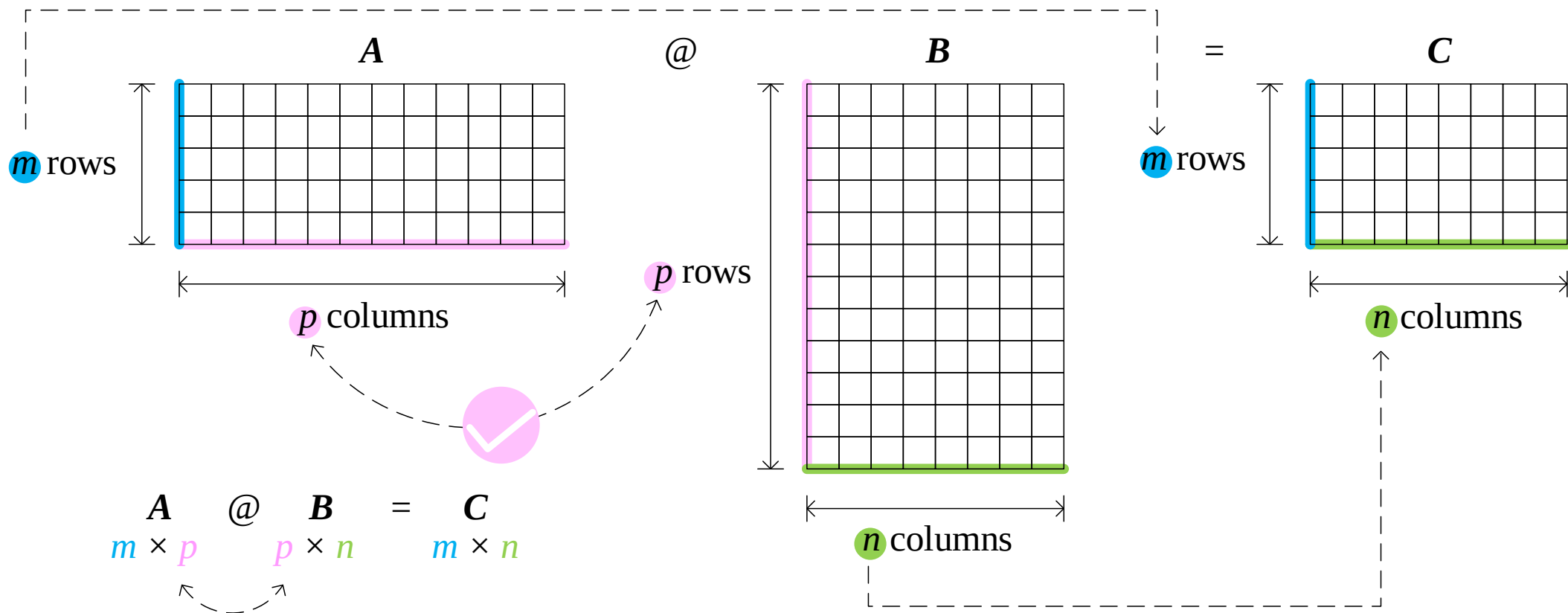


图2 • 矩阵乘积 $A @ B$

- 矩阵 A 和 B 的乘积 $C = AB = A @ B$ 是一个形状为 $m \times n$ 的矩阵
- 本书也会将矩阵乘法 AB 写成 $A @ B$, 与NumPy等Python库的语法保持一致



形状匹配规则：中间维度被“消去”

$(m \times p) @ (p \times n)$ 夹在中间的 p 被“消去”，乘积形状为 $m \times n$

- 乘积 C 继承了 A 的行数 m 、 B 的列数 n ，而 A 的列数与 B 的行数 p 相当于被约去
- 若左矩阵列数不等于右矩阵行数，则无法进行矩阵乘法运算

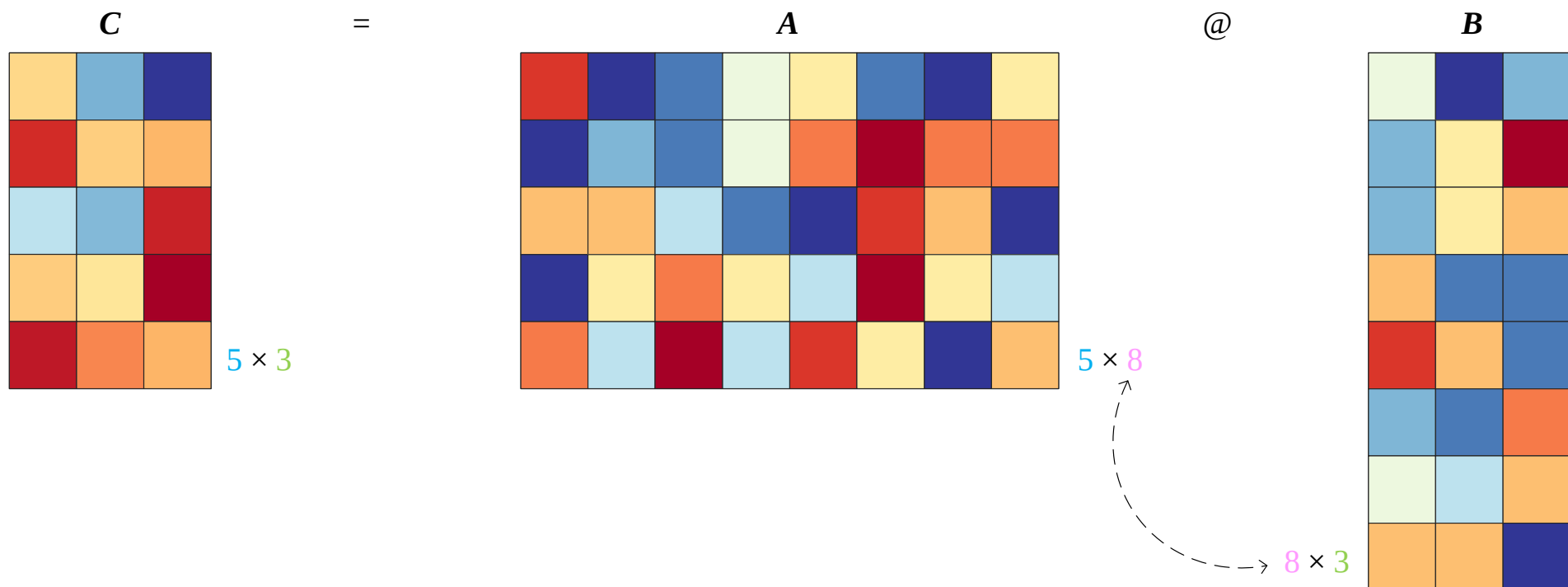
NumPy计算矩阵乘法

在NumPy中，矩阵乘法可通过@运算符、`np.matmul()` 或 `np.dot()` 函数实现

- 代码1采用@运算符完成矩阵乘法运算，并用热图可视化乘积结果
- $(5 \times 8) @ (8 \times 3)$ 夹在中间的8被“消去”，结果的形状为 (5×3)

图3 • 热图可视化矩阵乘积A @ B

- 用seaborn.heatmap()将矩阵C、A、B分别绘制为颜色矩阵，直观比较三者形状
- 颜色深浅代表数值大小，无需逐一读数即可把握矩阵的整体分布



热图可视化矩阵乘法

用 `@` 运算符完成矩阵乘法，并用热图并排展示 C 、 A 、 B

- `np.random.seed(88)` 固定随机种子，用 `np.random.randint()` 创建形状为 $(5,8)$ 的 A 和 $(8,3)$ 的 B
- `C = A @ B` 一步完成矩阵乘法，等价于 `np.matmul(A, B)`
- `plt.subplots(1,5,...)` 创建1行5列子图，依次放置 C 、等号、 A 、乘号、 B
- `sns.heatmap()` 的 `cmap`、`xticklabels`、`yticklabels`、`cbar_kws` 等参数控制热图外观

```
## 初始化
import numpy as np
import seaborn as sns
from matplotlib import pyplot as plt

## 创建矩阵A和B
np.random.seed(88)
A = np.random.randint(0, 10, (5, 8))
B = np.random.randint(0, 10, (8, 3))

## 矩阵乘法 A@B
C = A @ B

## 可视化矩阵乘法
fig, axs = plt.subplots(1, 5, figsize=(8, 3))

plt.sca(axs[0])
ax = sns.heatmap(C, cmap='RdYlBu_r',
                  xticklabels = [], yticklabels = [],
                  cbar_kws={"orientation": "horizontal"})
ax.set_aspect("equal")
plt.title('C')

plt.sca(axs[1])
plt.title('=') # 绘制等号
plt.axis('off')

plt.sca(axs[2])
ax = sns.heatmap(A, cmap='RdYlBu_r',
                  xticklabels = [], yticklabels = [],
                  cbar_kws={"orientation": "horizontal"})
ax.set_aspect("equal")
plt.title('A')

plt.sca(axs[3])
plt.title('@') # 绘制乘号
plt.axis('off')

plt.sca(axs[4])
ax = sns.heatmap(B, cmap='RdYlBu_r',
                  xticklabels = [], yticklabels = [],
                  cbar_kws={"orientation": "horizontal"})
ax.set_aspect("equal")
plt.title('B')
```

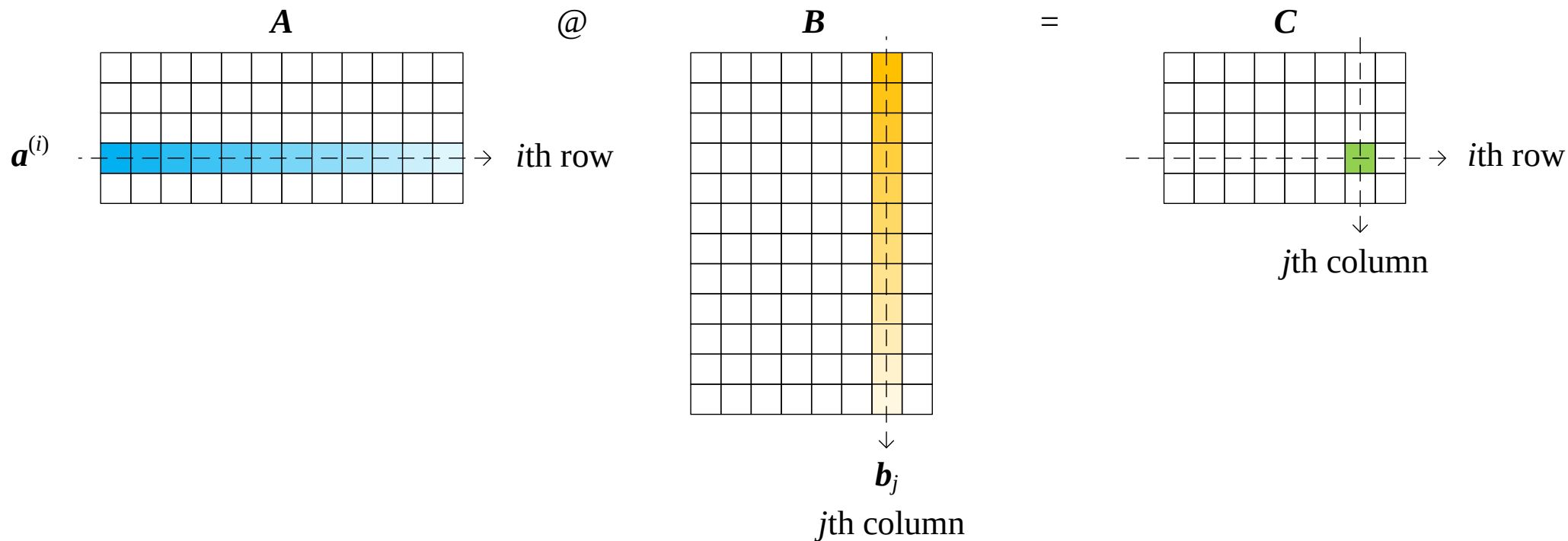
计算每个元素

矩阵 C 第 i 行、第 j 列的元素，等于 A 第 i 行与 B 第 j 列对应元素相乘后的累加和

- 这一求和过程，正是 A 的第 i 行行向量与 B 的第 j 列列向量的点积
- 矩阵乘法背后的核心运算，就是行向量与列向量的点积

图4 · 计算矩阵乘积 $C = AB$ 的每个元素

- 取 A 的第 i 行（高亮蓝色）与 B 的第 j 列（高亮橙色），对应元素相乘后累加，得到 C 中第 i 行第 j 列的元素（绿色）
- 遍历所有的 i 、 j 组合，就完成了整个矩阵乘法



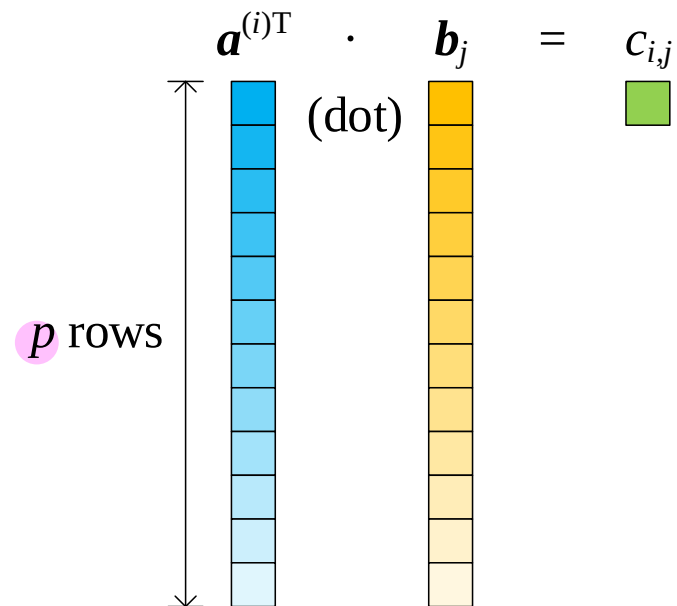
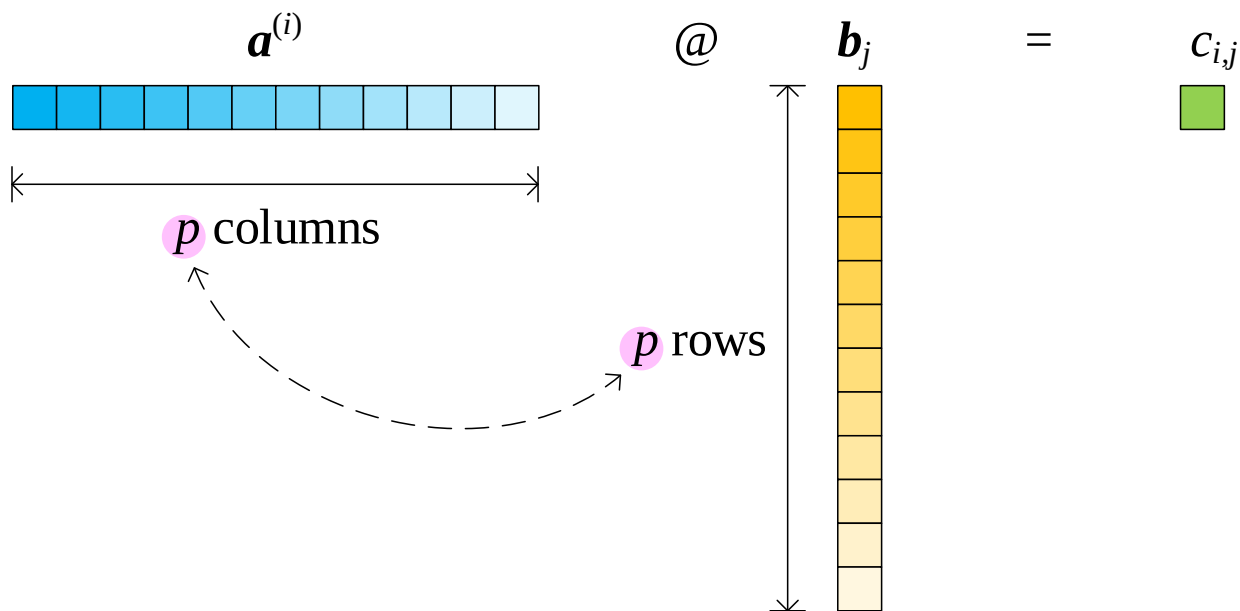
矩阵乘法的结果可以是标量

形状为 $1 \times p$ 的行向量与形状为 $p \times 1$ 的列向量相乘，结果是 1×1 矩阵，即一个标量

- 这一点特别值得注意：矩阵乘法未必总是产生“大矩阵”，也可能只是一个数
- 行向量与列向量的乘积，也可以写成向量内积（dot product）的形式

图5 • 矩阵乘积 $\mathbf{a}^{(i)} @ \mathbf{b}_j = \mathbf{c}_{i,j}$

- 形状为 $1 \times p$ 的行向量 $\mathbf{a}^{(i)}$ 与形状为 $p \times 1$ 的列向量 $\mathbf{b}_j$ 相乘，结果是一个标量
- 矩阵乘积 $\mathbf{C} = \mathbf{AB}$ 相当于完成了 $m \times n$ 个矩阵乘法，也是 $m \times n$ 个向量内积



举个例子

给定矩阵 $A = [1,2,3; 4,5,6]$, $B = [1,4; 2,5; 3,6]$

- A 的形状是 2×3 , B 的形状是 3×2 , 因此结果矩阵 $C = AB$ 的形状是 2×2
- $(2 \times 3) @ (3 \times 2)$ 夹在中间的3被“消去”, 乘积形状为 2×2

图6 · 矩阵乘积AB; A的形状是 2×3 , B的形状是 3×2

- 逐个元素计算: $c_{1,1}=1 \times 1 + 2 \times 2 + 3 \times 3 = 14$, $c_{1,2}=1 \times 4 + 2 \times 5 + 3 \times 6 = 32$
- $c_{2,1}=4 \times 1 + 5 \times 2 + 6 \times 3 = 32$, $c_{2,2}=4 \times 4 + 5 \times 5 + 6 \times 6 = 77$, 得到 $C = [14, 32; 32, 77]$

$$\begin{matrix} & \mathbf{A} & & @ & & \mathbf{B} & & = & & \mathbf{C} \\ \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline \end{array} & & & & & \begin{array}{|c|c|} \hline 1 & 4 \\ \hline 2 & 5 \\ \hline 3 & 6 \\ \hline \end{array} & & & & \begin{array}{|c|c|} \hline 14 & 32 \\ \hline 32 & 77 \\ \hline \end{array} \end{matrix}$$

图7 · 计算矩阵乘积AB的每个元素

- $\mathbf{a}^{(1)} @ \mathbf{b}_1 = 14$, $\mathbf{a}^{(1)} @ \mathbf{b}_2 = 32$, 对应 $\mathbf{C}$ 的第一行
- $\mathbf{a}^{(2)} @ \mathbf{b}_1 = 32$, $\mathbf{a}^{(2)} @ \mathbf{b}_2 = 77$, 对应 $\mathbf{C}$ 的第二行

$$\mathbf{a}^{(1)} @ \mathbf{b}_1 = c_{1,1}$$

1	2	3
4	5	6

1	4
2	5
3	6

14	32
32	77

$$\mathbf{a}^{(1)} @ \mathbf{b}_2 = c_{1,2}$$

1	2	3
4	5	6

1	4
2	5
3	6

14	32
32	77

$$\mathbf{a}^{(2)} @ \mathbf{b}_1 = c_{2,1}$$

1	2	3
4	5	6

1	4
2	5
3	6

14	32
32	77

$$\mathbf{a}^{(2)} @ \mathbf{b}_2 = c_{2,2}$$

1	2	3
4	5	6

1	4
2	5
3	6

14	32
32	77

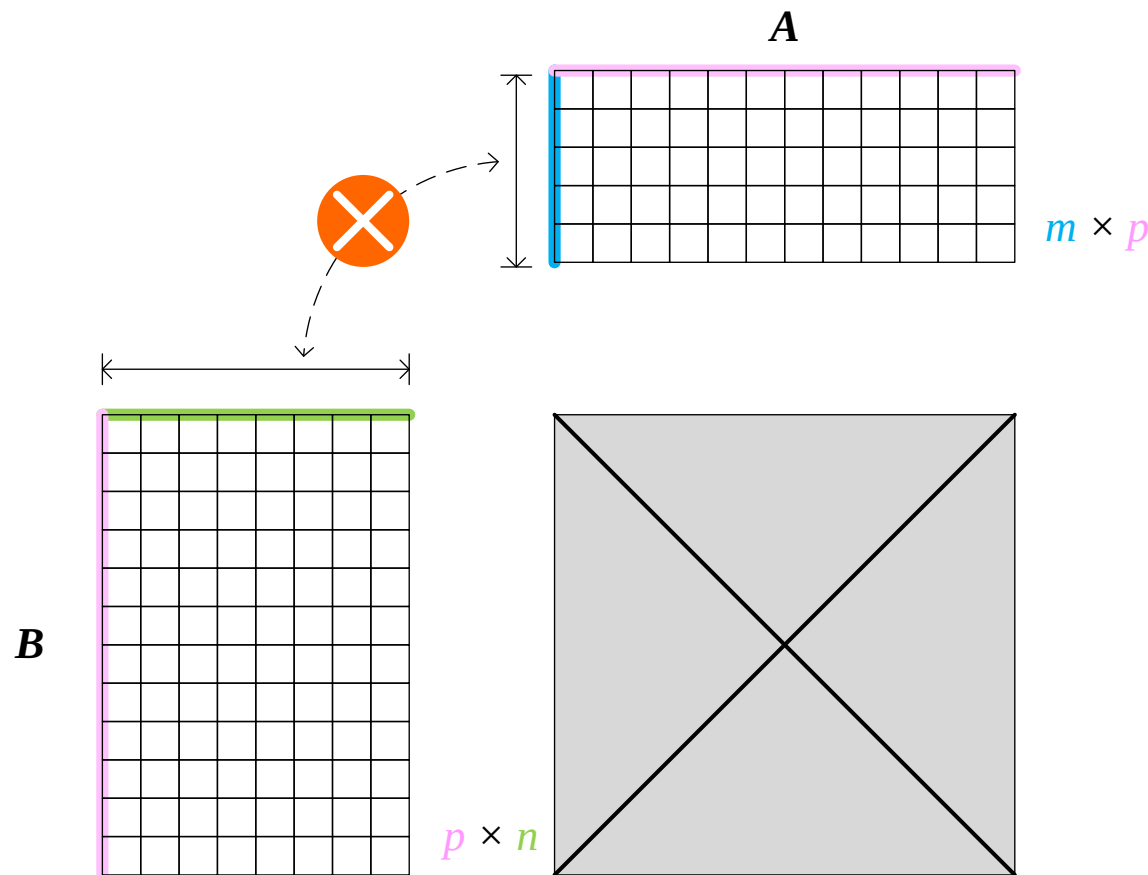
矩阵乘法不满足交换律

矩阵乘法的左右位置非常重要，不能随意交换： $AB \neq BA$ (*non-commutative*)

- $A @ B$ 满足矩阵乘法运算法则，不代表 $B @ A$ 也同样满足
- 即便 $B @ A$ 也满足运算法则，也不意味着 $A @ B$ 和 $B @ A$ 的结果相同

图8 • $B @ A$ 不满足矩阵乘法运算原则

- A 形状为 $m \times p$, B 形状为 $p \times n$; $B @ A$ 要求 B 的列数 n 等于 A 的行数 m
- 若 $n \neq m$, $B @ A$ 根本无法进行, 形状匹配是矩阵乘法的第一道门槛



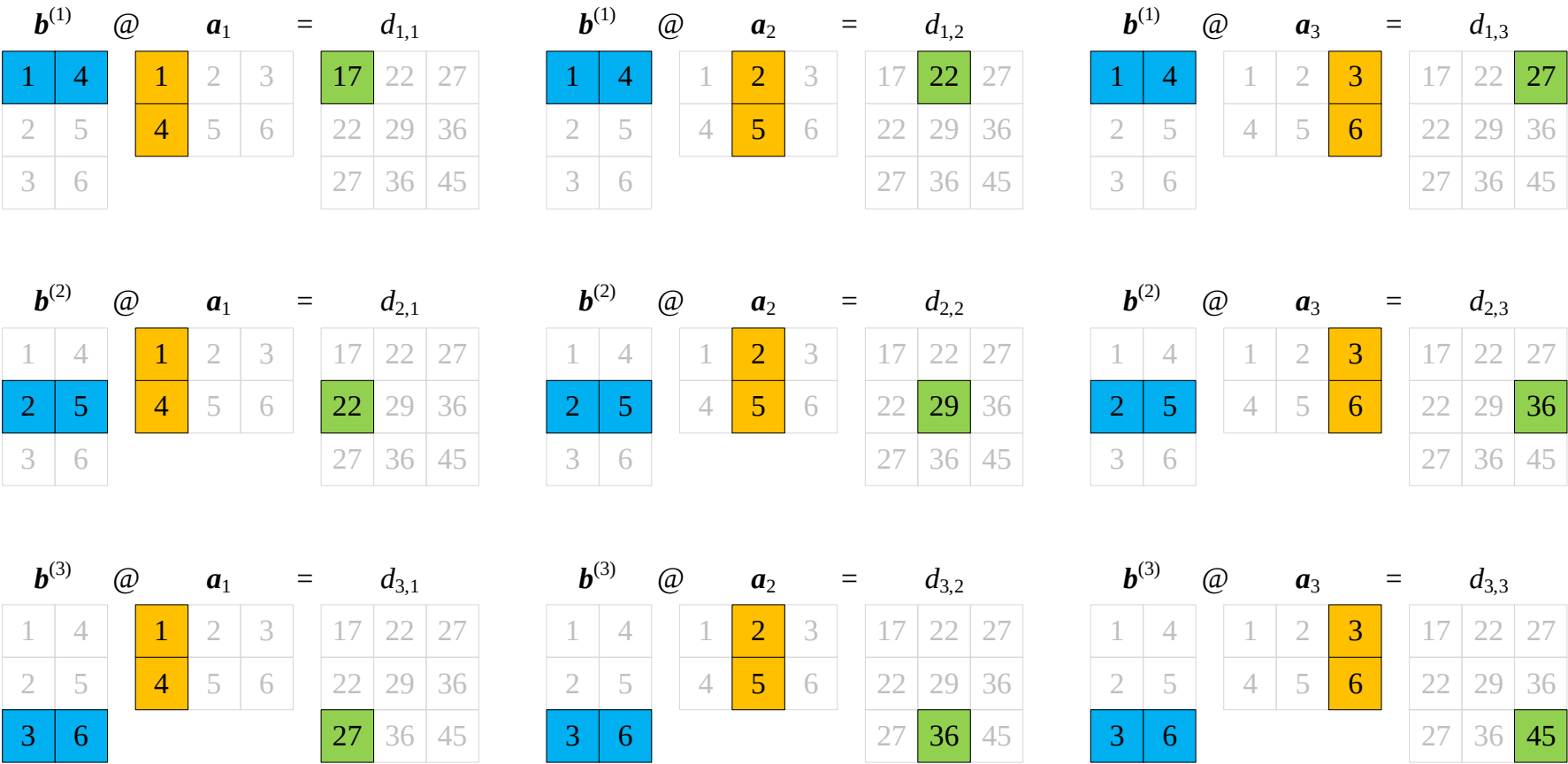
B @ A的结果显然不同于A @ B

$$\mathbf{D} = \mathbf{B} @ \mathbf{A} = [1,4; 2,5; 3,6] @ [1,2,3; 4,5,6] = [17,22,27; 22,29,36; 27,36,45]$$

- $(3 \times 2) @ (2 \times 3)$ 夹在中间的2被“消去”， $\mathbf{D}$ 的形状为 3×3
- $\mathbf{D}$ 的形状与 $\mathbf{C} = \mathbf{A}\mathbf{B}$ 的形状 (2×2) 完全不同，直观说明 $\mathbf{AB} \neq \mathbf{BA}$

图9 · 计算矩阵乘积B @ A的每个元素

- 逐格计算D的9个元素，每个元素都是B的一行与A的一列的点积
- 细心的读者会发现，AB与BA在这个例子中都恰好是对称矩阵



自定义Python函数计算矩阵乘法

用三层for循环手动实现矩阵乘法，帮助理解@运算符背后的计算过程

- 外层遍历 A 的行、中层遍历 B 的列、内层做对应元素相乘再累加
- 若 A 的列数与 B 的行数不匹配，函数会主动抛出错误提示

自定义函数：三层for循环实现矩阵乘法

用 `matrix_multiplication(A, B)` 手动复现矩阵乘法的计算过程

- `m, p_A = A.shape` 与 `p_B, n = B.shape` 取出两个矩阵的形状
- `if p_A != p_B` 检测形状是否匹配，不匹配则 `raise ValueError` 抛出异常
- `C = np.zeros((m, n))` 创建全0结果矩阵，形状为 `(m, n)`
- 三层for循环 `i, j, k` 依次遍历行、列、对应求和维度，`C[i,j] += A[i,k]*B[k,j]`

```
## 初始化
import numpy as np

## 自定义函数
def matrix_multiplication(A, B):

    # 获取矩阵 A 和 B 的形状
    m, p_A = A.shape
    p_B, n = B.shape

    # 检测矩阵形状是否符合矩阵乘法规则
    if p_A != p_B:
        raise ValueError('Dimensions do not match')

    # 初始化结果矩阵 C，形状 (m, n)，初始值设为 0
    C = np.zeros((m, n))

    # 进行矩阵乘法计算，使用三层 for 循环
    for i in range(m):          # 遍历 A 的行
        for j in range(n):      # 遍历 B 的列
            for k in range(p_A): # 遍历 A 的列 / B 的行
                C[i, j] += A[i, k] * B[k, j] # 逐元素累加

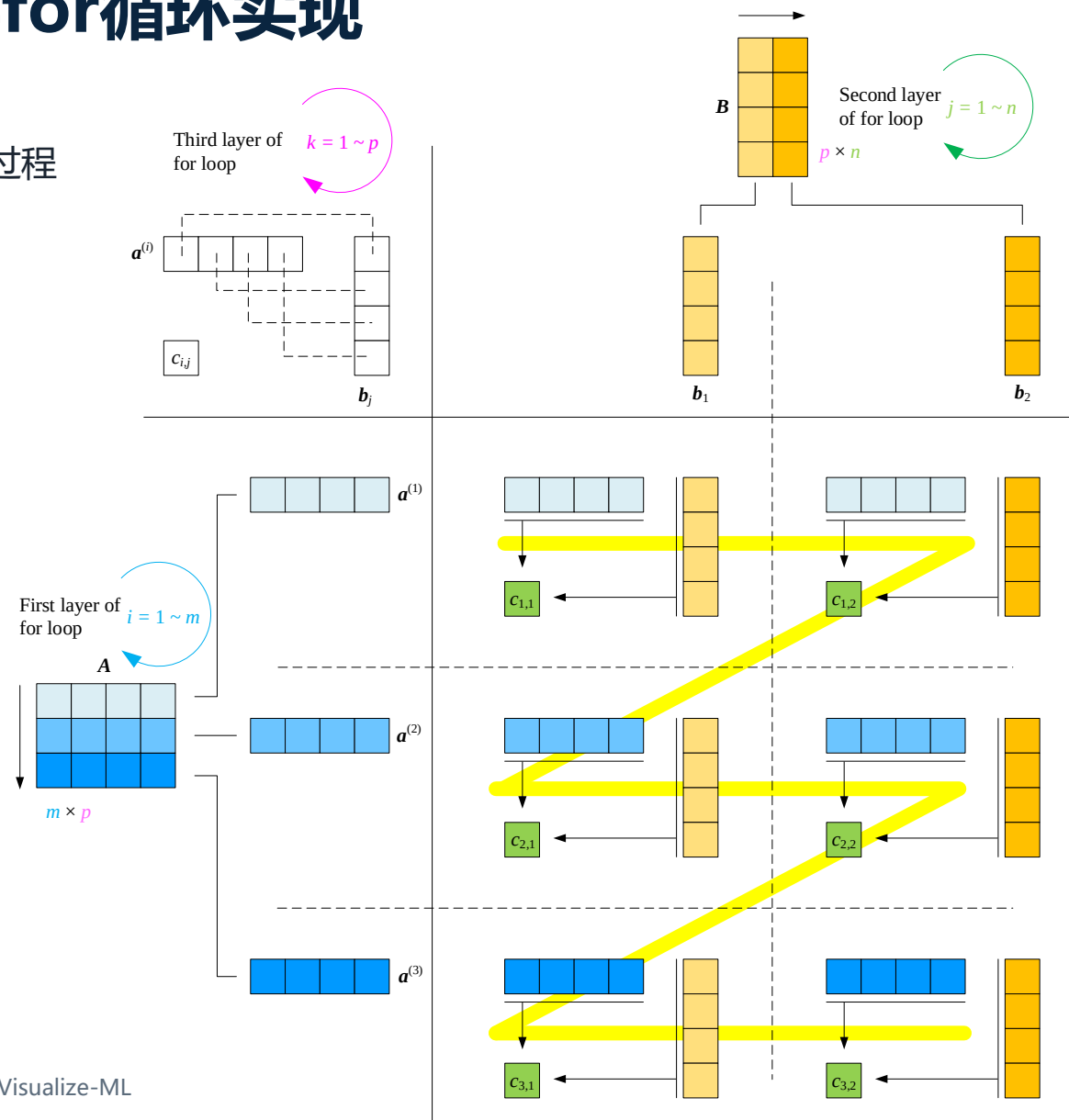
    return C

## 定义矩阵
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = A.T

## 矩阵乘法
matrix_multiplication(A, B)
matrix_multiplication(B, A)
```

图10 • 矩阵乘法AB规则，利用三层for循环实现

- 第一层循环i遍历A的每一行，第二层循环j遍历B的每一列
- 第三层循环k对应元素相乘再累加，正是行向量与列向量的点积过程



矩阵乘法的重要性怎么强调都不为过

本节仅展示了矩阵乘法世界的冰山一角，形状匹配与逐元素点积是理解一切的起点

- 矩阵乘法是数据科学、机器学习中最基础也最核心的运算之一
- 熟练掌握@运算符与三层for循环的对应关系，有助于理解更复杂的矩阵运算

本节关键术语

矩阵乘法 *Matrix Multiplication* — 两矩阵相乘生成新矩阵, 记作 AB 或 $A @ B$

点积 *Dot Product* — 行向量与列向量对应元素相乘后求和

结合律 *Associative Law* — 矩阵乘法满足 $(AB)C = A(BC)$

标量结果 *Scalar Result* — 1×1 矩阵的乘积可视作一个普通数字

热图可视化 *Heatmap* — 用颜色深浅展示矩阵数值分布的可视化方式

形状匹配规则 *Shape Compatibility* — 左矩阵列数须等于右矩阵行数

交换律 *Commutative Law* — 矩阵乘法一般不满足, 即 $AB \neq BA$

分配律 *Distributive Law* — 矩阵乘法满足 $A(B+C) = AB + AC$

@ 运算符 *NumPy* 中执行矩阵乘法的简洁写法, 等价于 `np.matmul()`

三层for循环 *Triple Nested Loop* — 手工实现矩阵乘法的底层算法逻辑

配套代码与延伸阅读

延伸阅读 • LA_02_04_02.ipynb

逐元素计算矩阵乘法的完整代码，建议自学

延伸阅读 • LA_02_04_03.ipynb

自定义Python函数、三层for循环实现矩阵乘法

推荐阅读 • 《矩阵力量》

深入理解矩阵乘法在数据科学、机器学习中的应用场景

开源仓库

全书代码与图像素材：<https://github.com/Visualize-ML>

习题 Q1 – Q9

Q1 判定如下成对矩阵的形状是否能够完成 $A @ B$ 或 $B @ A$ 矩阵乘法： $A_{2 \times 2} = [1, 2; 3, 4]$ 与 $B_{2 \times 2} = [5, 6; 7, 8]$ ； $A_{1 \times 3} = [1, 2, 3]$ 与 $B_{3 \times 1} = [4; 5; 6]$ ； $A_{3 \times 2} = [1, 2; 3, 4; 5, 6]$ 与 $B_{2 \times 2} = [7, 8; 9, 10]$ ； $A_{2 \times 3} = [1, 2, 3; 4, 5, 6]$ 与 $B_{3 \times 2} = [7, 8; 9, 10; 11, 12]$ ； $A_{2 \times 2} = [1, 2; 3, 4]$ 与 $B_{2 \times 1} = [5; 6]$ 。

Q3 给定列向量 $a = [1; 2]$ ，计算 $a^T @ a$ 、 $a @ a^T$ 。

Q5 给定 $A = [1, 2; 3, 4]$ 和 $b = [1; 2]$ ，计算 $A @ b$ 、 $b^T @ A^T$ 。

Q7 请用NumPy计算图9中每个子图对应的矩阵乘法，并且用向量内积演算验证。

Q9 用NumPy随机生成 $A(4 \times 6)$ 和 $B(6 \times 3)$ 矩阵，验证 $A @ B$ 的形状是否符合“中间维度被消去”的规则。

Q2 如下10组成对矩阵形状都同时支持 $A @ B$ 和 $B @ A$ ，请通过合适的办法（手算、编程）判断 $A @ B$ 和 $B @ A$ 是否相等，包括单位矩阵、对称矩阵、全1矩阵、对角矩阵与旋转矩阵等情形。

Q4 给定列向量 $a = [1; 2]$ 和 $b = [1; 2; 3]$ ，计算 $a @ b^T$ 、 $b @ a^T$ 。

Q6 给定 $A = [1, 2; 3, 4; 5, 6]$ ，计算 $A @ A^T$ 、 $A^T @ A$ 。

Q8 修改代码2，交换第1、2层for循环，计算矩阵乘法，观察结果是否发生变化。

习题 Q10 – Q19

Q10 编写函数`check_shape(A, B)`，判断能否进行 $A @ B$ 运算，能则返回结果矩阵的形状，否则给出提示。

Q12 用NumPy验证矩阵乘法对加法满足分配律： $A @ (B + C)$ 与 $A @ B + A @ C$ 是否相等。

Q14 编写函数`dot_product_check(A, B)`，用行向量与列向量的点积逐个验证 $A @ B$ 每个元素的正确性。

Q16 用`seaborn.heatmap()`绘制一个非对称矩阵 A 与其转置 A^T 的热图，比较二者的差异。

Q18 用NumPy验证：对角矩阵 D 与任意同阶方阵 A 相乘， $D @ A$ 是否等价于对 A 的每一行做缩放。

Q11 用NumPy验证矩阵乘法满足结合律： $(A @ B) @ C$ 与 $A @ (B @ C)$ 是否相等。

Q13 生成随机方阵 A ，验证 $A @ I$ 与 $I @ A$ 是否都等于 A 本身（ I 为同阶单位矩阵）。

Q15 用Python分别计时NumPy的`@`运算符与自定义三层for循环函数，比较大矩阵（如 200×200 ）下二者的运行速度差异。

Q17 给定一个方阵 A ，验证 $A @ A$ （即 A^2 ）与 $A @ A @ A$ （即 A^3 ）的形状是否始终保持不变。

Q19 生成两个 2×2 旋转矩阵，验证两次旋转矩阵相乘的结果，是否等价于一次合并角度的旋转。

习题 Q20 – Q28

Q20 编写函数`is_result_symmetric(A)`，验证 $A @ A^T$ 和 $A^T @ A$ 是否恒为对称矩阵，并用随机矩阵反复验证。

Q22 生成一个 5×5 随机矩阵 A ，计算 $A @ A$ 的迹（trace），并与 A 特征值之和比较（可用`numpy.linalg.eigvals()`）。

Q24 用`seaborn.heatmap()`将本节两个例子（ $A @ B$ 与 $B @ A$ ）并排可视化，直观比较形状与数值差异。

Q26 用NumPy验证零矩阵 O 与任意矩阵 A 相乘， $A @ O$ 和 $O @ A$ 是否都等于形状匹配的零矩阵。

Q28 编写一个“矩阵乘法可视化器”：输入两个小矩阵，依次打印每一步 c_{ij} 对应的行向量·列向量计算过程。

Q21 用NumPy比较`np.matmul()`、`np.dot()`、`@`运算符在二维矩阵乘法上的结果是否完全一致。

Q23 编写函数`batch_matmul(A_list, B_list)`，输入两组矩阵列表，返回对应位置矩阵乘积组成的列表。

Q25 给定长方形矩阵 A （ $m \times p$ ， $m \neq p$ ），比较 $A @ A^T$ 与 $A^T @ A$ 的形状差异，并说明二者分别常用于什么场景（如协方差矩阵）。

Q27 编写函数`matmul_report(A, B)`，自动判断 $A @ B$ 、 $B @ A$ 是否都可行，若可行则输出各自形状，并提示二者是否相等。

10 个可深入探讨的 Prompt (1/2)

- 1 请解释矩阵乘法形状匹配规则 “中间维度被消去” 背后的数学原因，并联系点积的定义。
- 2 为什么矩阵乘法的结果可以是 1×1 矩阵（标量）？这与向量内积有什么关系？
- 3 请证明矩阵乘法一般不满足交换律，并举一个 $AB=BA$ 成立的特殊例子（如单位矩阵）。
- 4 请解释三层for循环实现矩阵乘法的时间复杂度，以及为什么NumPy的@运算符比手写循环快得多。
- 5 请设计一个Python实验，验证矩阵乘法满足结合律 $(AB)C=A(BC)$ 。

10 个可深入探讨的 Prompt (2/2)

6 请解释为什么在机器学习中，矩阵乘法常用于批量计算（例如全连接层的前向传播）。

7 请用几何视角解释旋转矩阵相乘为什么等价于角度相加。

8 请解释 AA^T 和 A^TA 为什么总是对称矩阵，并说明它们在统计学（如协方差矩阵）中的应用。

9 如果调换自定义函数中三层for循环的顺序（i、j、k），结果是否会变化？运行速度是否会变化？请解释原因。

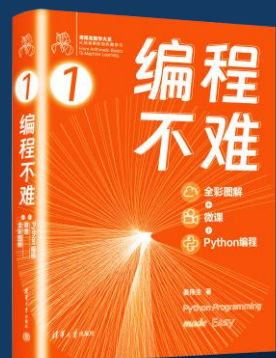
10 请解释热图可视化矩阵乘法的意义，以及如何通过颜色变化直观理解矩阵乘法的结果。

数学

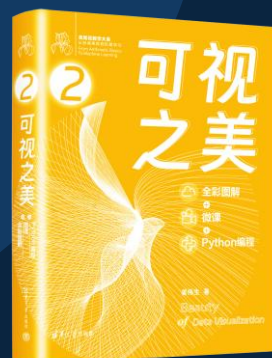
数学基础

线性代数

概率统计

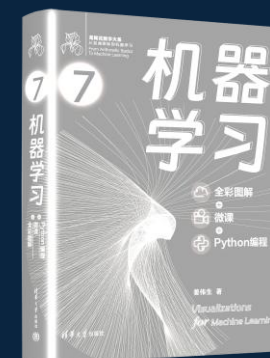
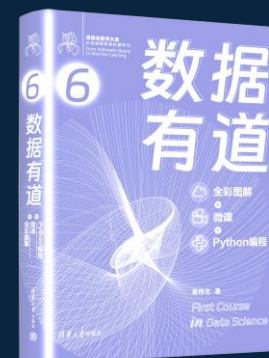
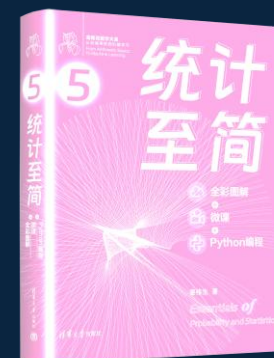


Python编程



数据可视化

工具



数据分析
图和网络

回归、降维
分类、聚类

实践

